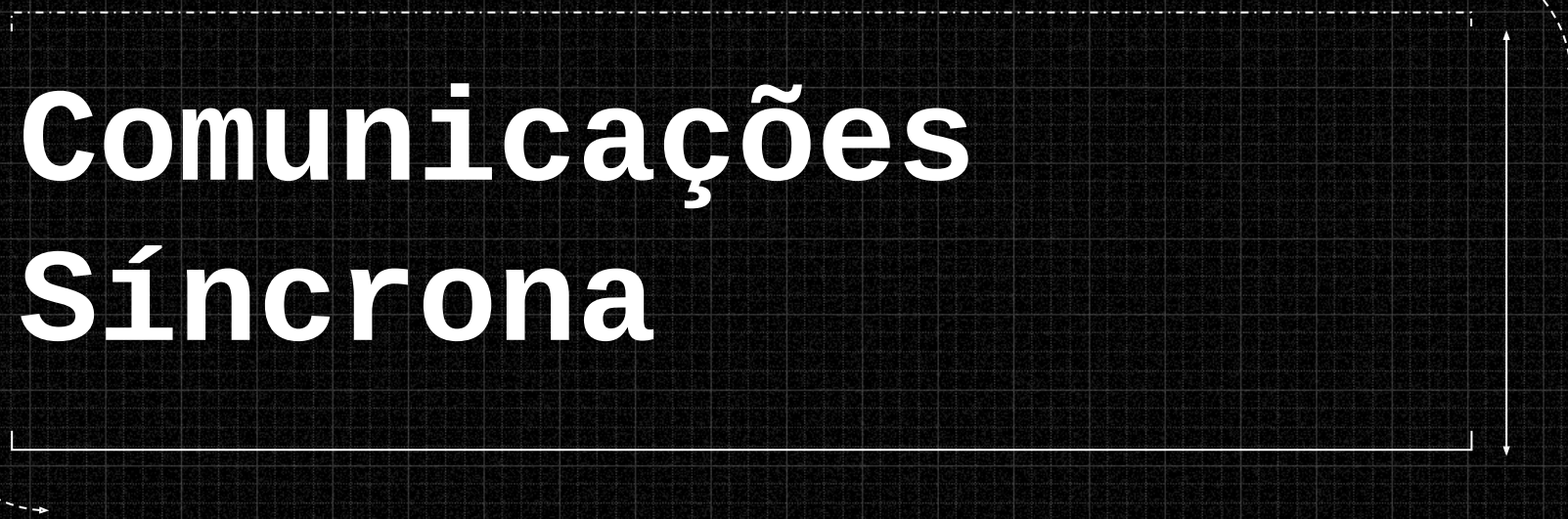


System Design

Comunicações Síncrona

A dashed rectangular box encloses the text 'Comunicações Síncrona'. A dashed arrow starts at the top right corner of the box, curves downwards and to the left, pointing towards the top left corner. Another dashed arrow starts at the bottom left corner of the box, curves upwards and to the right, pointing towards the bottom right corner. A solid vertical arrow points upwards from the bottom right corner of the box towards the top right corner.

\$ whoami

Matheus Fidelis

Engenheiro de \$RANDOM

@fidelissauro

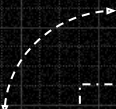
<https://fidelissauro.dev>

<https://linktr.ee/fidelissauro>



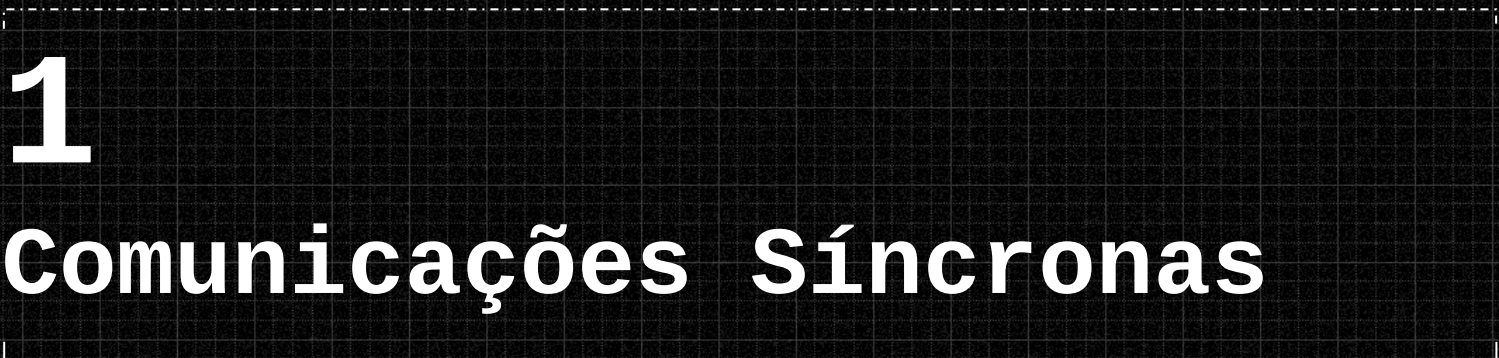
OBJETIVOS

- ✔ Continuação de protocolos e redes
- ✔ Arquiteturas distribuídas modernas
- ✔ REST, Webhooks, RPC/gRPC, WebSockets e GraphQL



1

Comunicações Síncronas



Definições e
Conceitos

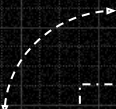


Comunicações Sincronas

- Cliente espera a resposta antes de continuar o fluxo
- Natureza bloqueante e dependente da disponibilidade do servidor
- Consistência e sequência são prioritárias
- Cada requisição é um diálogo imediato

Tradeoffs e Cenários

- Interações determinísticas
- Na comunicação síncrona, cada requisição é um diálogo imediato
- Previsibilidade, mas também acoplamento e latência.
- Fácil implementação e depuração
- Consistência transacional
- Baixa escalabilidade sob alta concorrência
- Falhas em cascata e degradação em cadeia



2

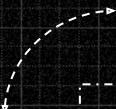
HTTP

Definições e
Conceitos



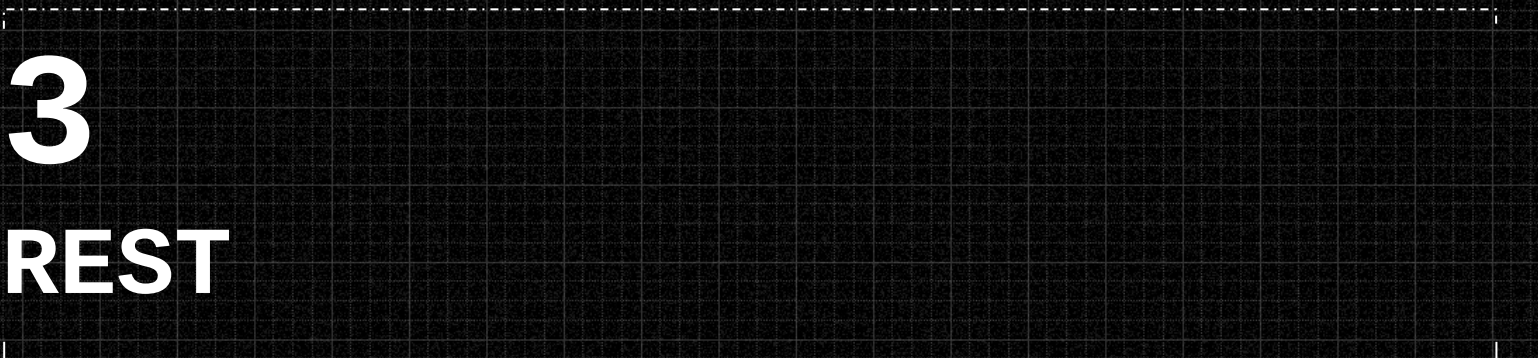
HTTP - Hypertext Transfer Protocol

- Camada 7 do Modelo OSI
- Construído em TCP/IP
- Base de Sistemas Web
- Request/Response
- Cliente/Servidor
- Protocolos Síncronos
- Body, Headers, Cookies, Headers, Status Codes
- Base para o REST

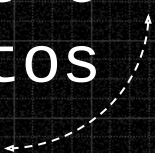


3

REST



Definições e
Conceitos



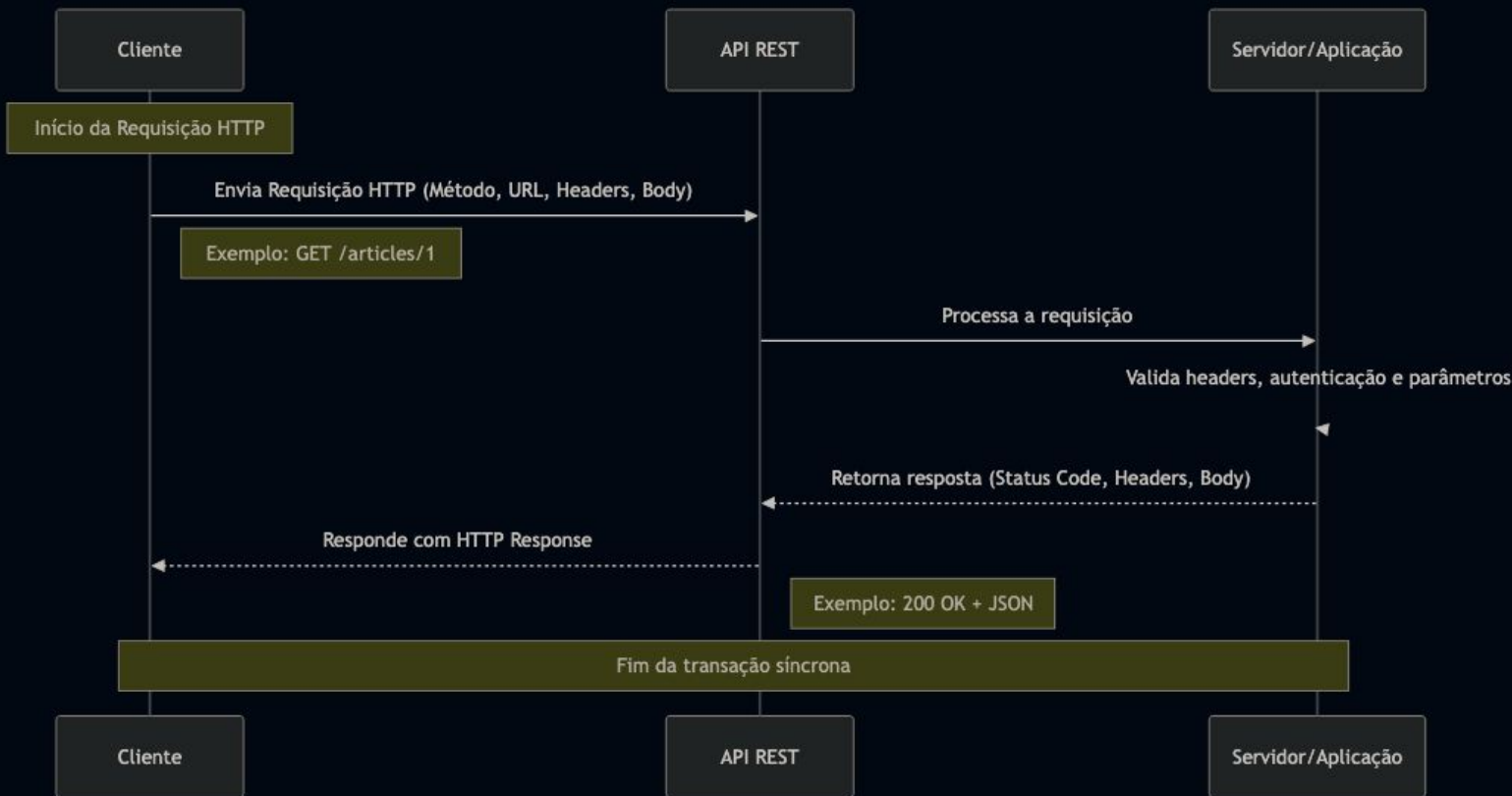
REST (Representational State Transfer)

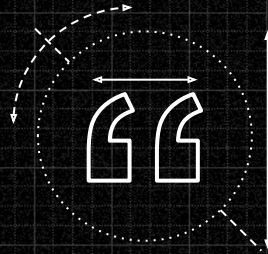
- Principal modelo de comunicação síncrona entre microserviços
- Estilo arquitetural proposto por Roy Fielding (2000)
- Baseado em princípios HTTP: recursos, URIs e verbos (GET, POST...)
- Interpretação dos recursos HTTP
- Simplicidade e compatibilidade universal

Estrutura de uma Requisição REST

- Client/Server
- Componentes: URI, headers, query strings e body
- Uso semântico de métodos HTTP
- Importância dos headers (Content-Type, Authorization, Cache-Control)
- Intenção e contexto pelo próprio request

Estrutura de uma Requisição REST



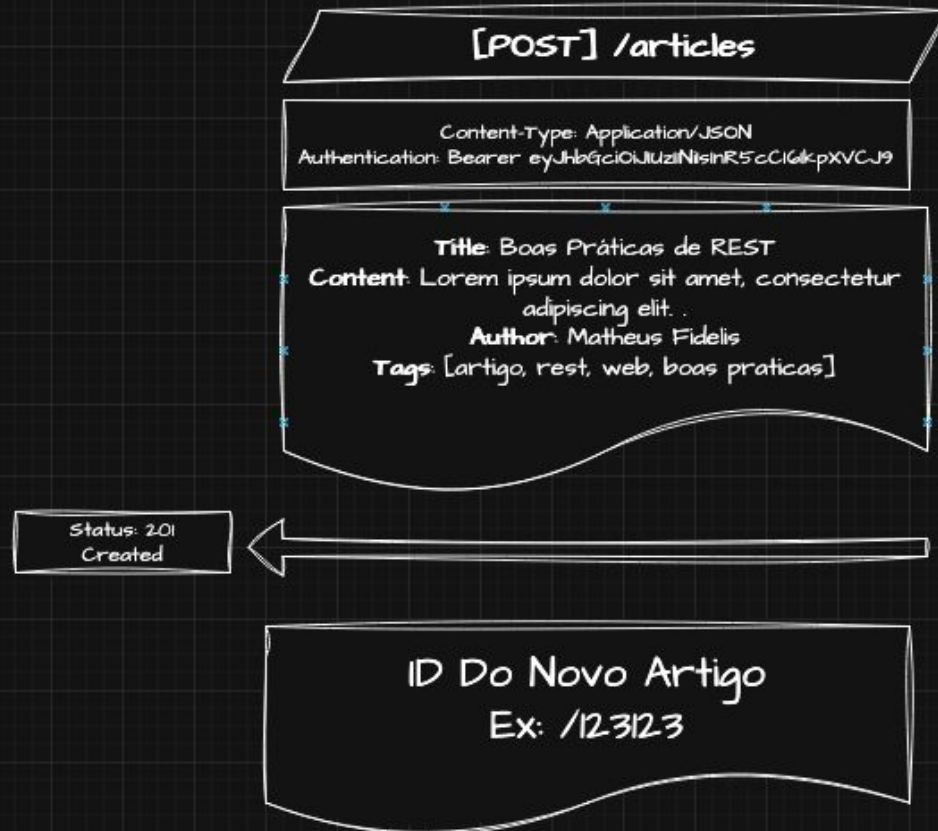


**"Estilo arquitetural, não um
protocolo"**

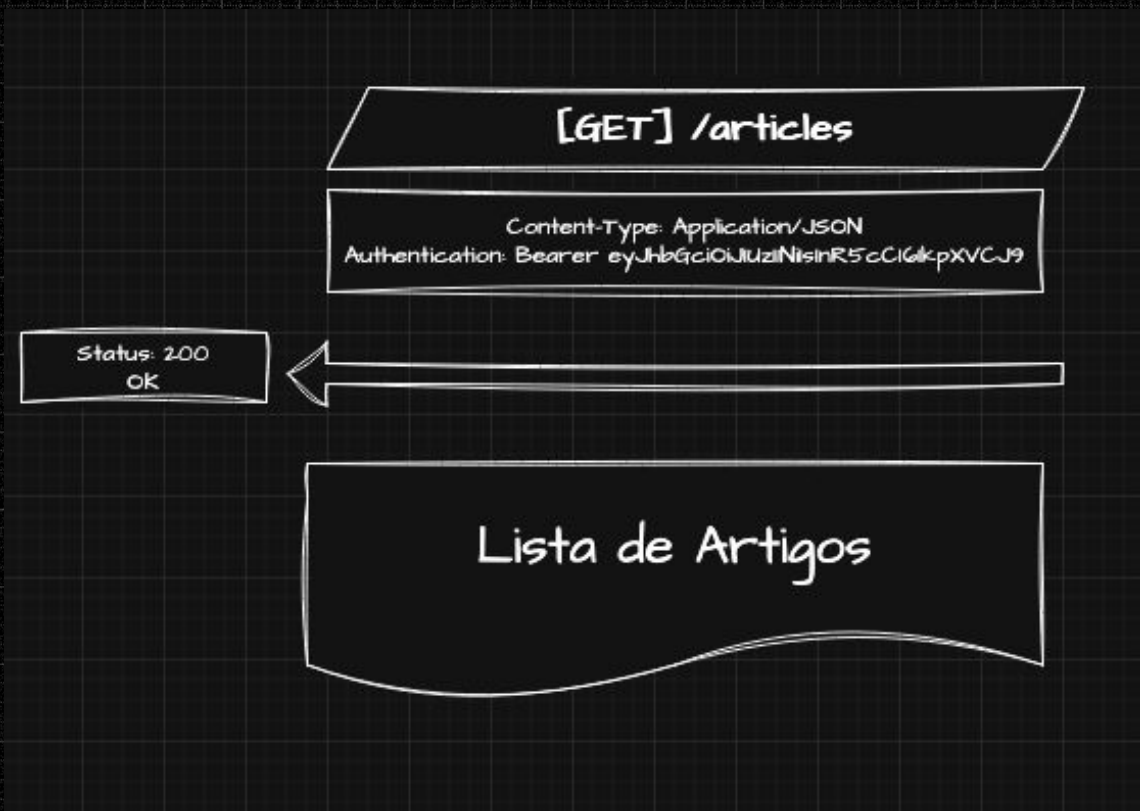
Princípios Arquiteturais do REST

- Interface Uniforme
- Representação das entidades
- Arquitetura em Camadas
- Desacoplamento
- Interoperabilidade entre sistemas
- Uso de cache e controle de validade (ETag, Cache-Control)
- Operações por Verbos

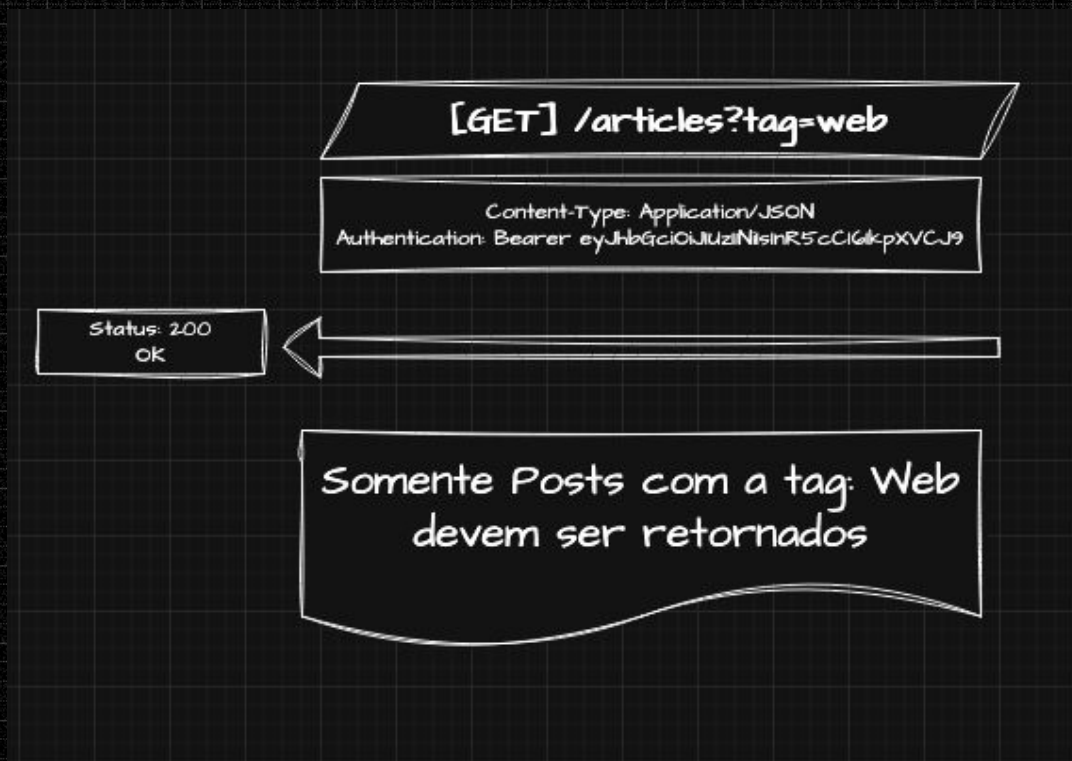
POST - Criar



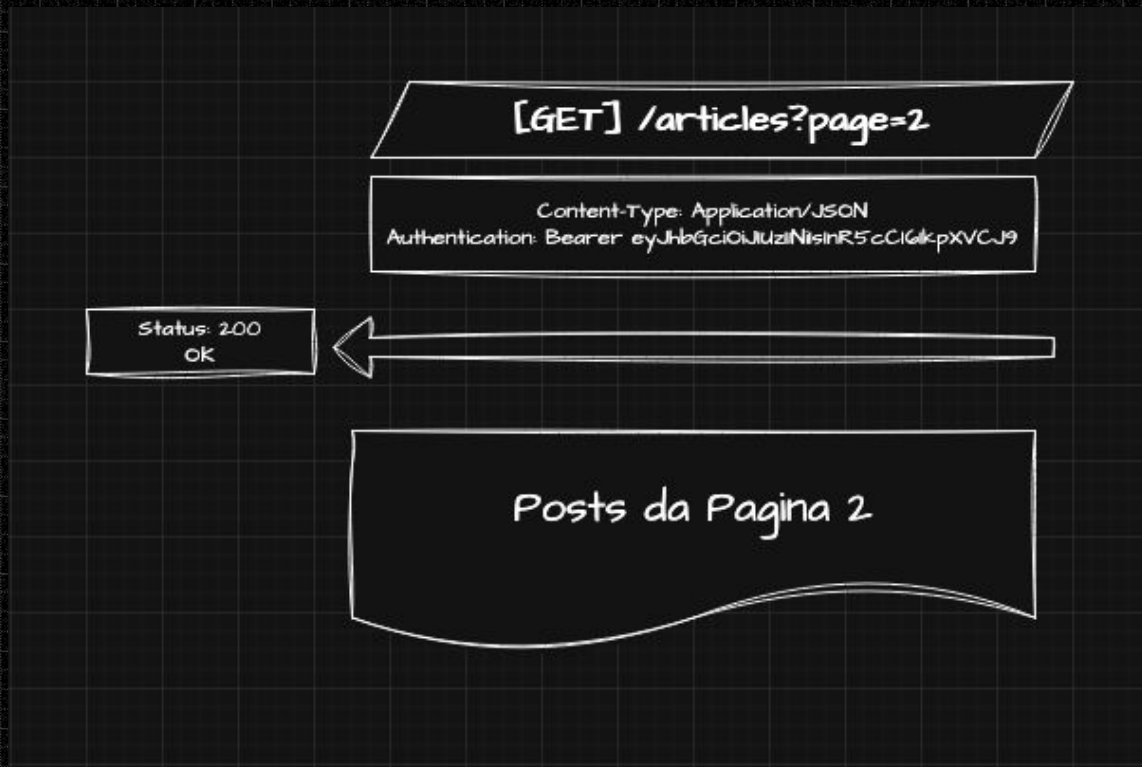
GET - Consultas - Lista



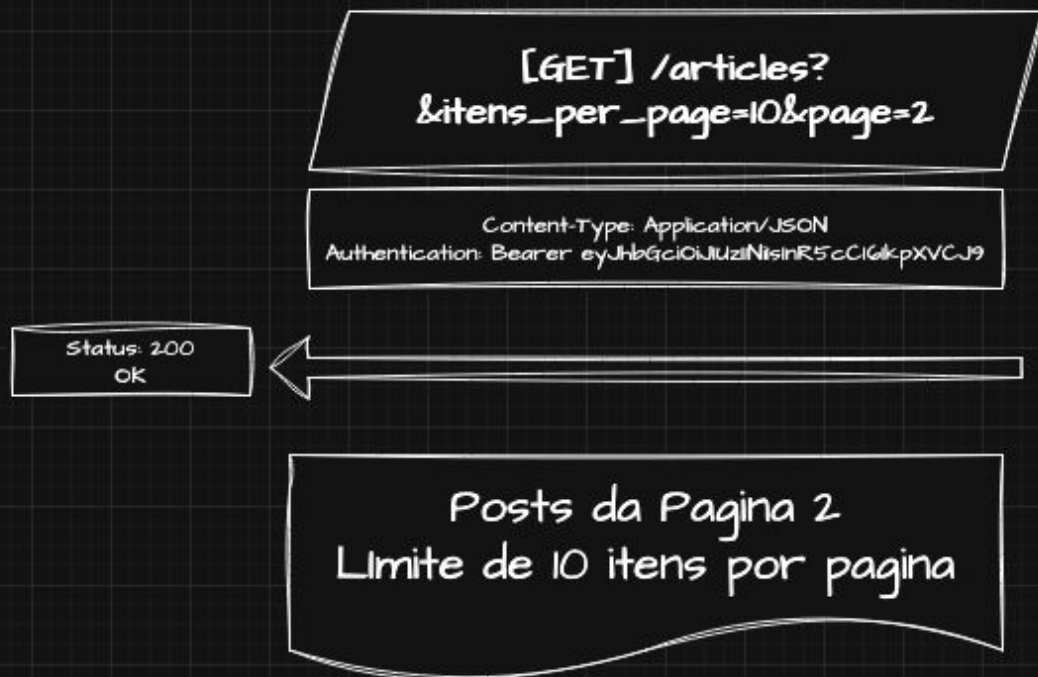
GET - Consultas - Filtros



GET - Consultas - Paginação



GET - Consultas - Paginação



PUT - Editar / Substituir

[PUT] /articles/123123

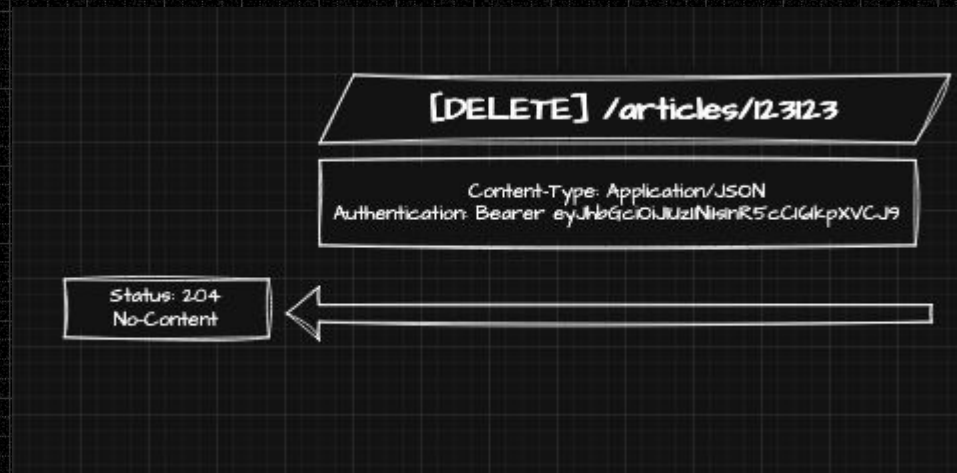
Content-Type: Application/JSON
Authentication: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

Title: Boas Práticas de REST v2
Content: Lorem ipsum dolor sit amet, consectetur
adipiscing elit. .
Author: Matheus Fidelis
Tags: [artigo, rest, web, boas praticas.]

Status: 200
OK

ID Artigo Atualizado
Ex: /123123

DELETE - Deletar



Idempotencia e Métodos HTTP

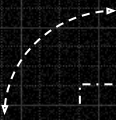
- Requisições repetidas não devem gerar efeitos colaterais
- Todo request gera o mesmo resultado
- Garante previsibilidade em retries
- propriedade é essencial em arquiteturas resilientes com retry e fallback.

Idempotencia e Métodos HTTP

- GET, PUT, PATCH e DELETE devem ser naturalmente idempotentes
- POST exige mecanismos adicionais (chaves idempotentes)

Idempotencia e Métodos HTTP

Método	Descrição	Idempotência
GET	Utilizado para recuperar a representação de um recurso sem modificá-lo. É seguro e idempotente, várias requisições idênticas devem ter o mesmo efeito que uma única requisição.	Sim
POST	Empregado para criar um novo recurso. Não é idempotente, pois realizar várias requisições POST pode criar múltiplos recursos. Em caso de necessidade de idempotência, será necessário a implementação de lógicas adicionais.	Não
PUT	Utilizado para atualizar um recurso existente ou criar um novo se ele não existir, no URI especificado. É idempotente, então múltiplas requisições idênticas terão o mesmo efeito sobre a entidade.	Sim
PATCH	Utilizado para aplicar atualizações parciais a um recurso. Ao contrário do PUT, que substitui o recurso inteiro, o PATCH modifica apenas as partes especificadas. É idempotente, pois a execução sob o mesmo recurso tende a gerar sempre o mesmo efeito e gerar o mesmo resultado.	Sim
DELETE	Empregado para remover um recurso. É idempotente, pois deletar um recurso várias vezes tem o mesmo efeito que deletá-lo uma única vez.	Sim



4

Webhooks e Pooling

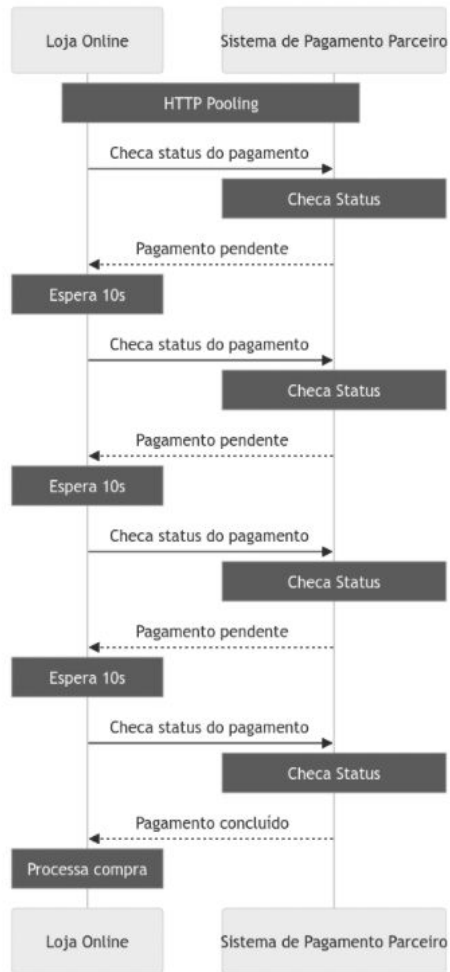


Definições e
Conceitos



Pooling

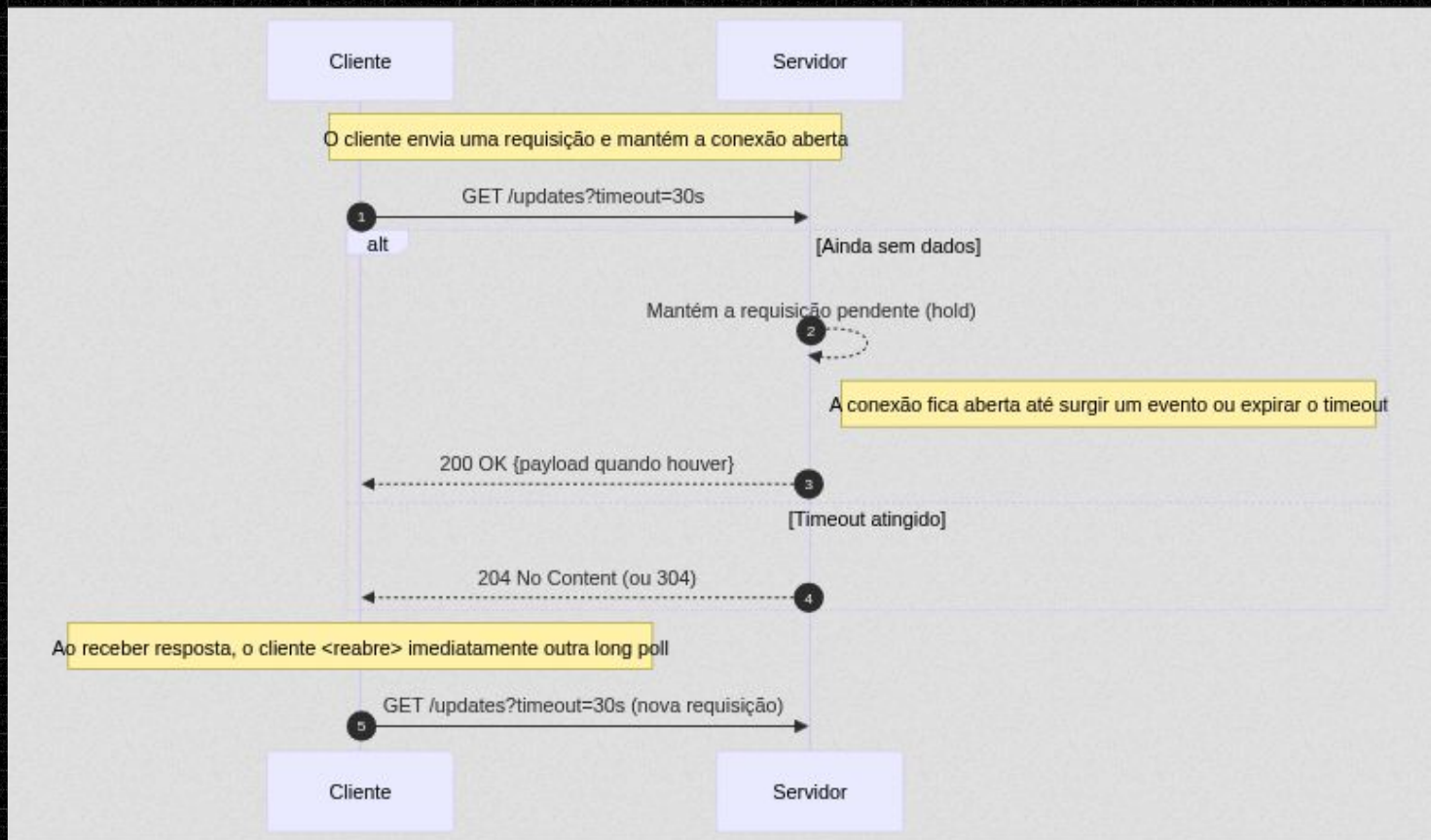
- Pooling é uma técnica usada quando não temos canais reativos nativos
- Iniciação Temporária (Status 202: Accepted)
- Check periódico do estado do novo recurso
- Insistencia até a conclusão
- Necessário Timeouts para evitar sobrecarga
- Short Pooling e Long Pooling



Long-Pooling

- O cliente envia uma requisição e mantém a conexão aberta até que o servidor tenha novos dados.
- Quando há uma atualização, o servidor responde e o cliente imediatamente abre uma nova requisição.
- Reduz a quantidade de requisições ociosas e melhora o tempo de resposta perceptível.
- Exige time-outs bem configurados e gestão de conexões persistentes.

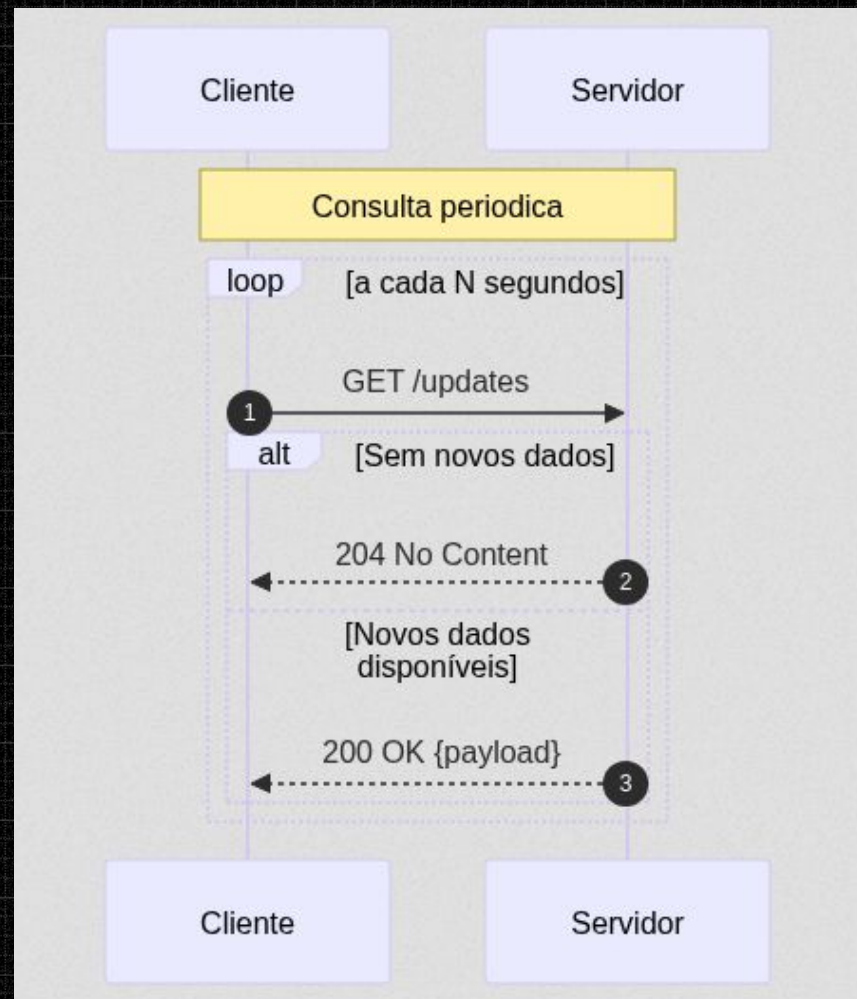
Long-Pooling



Short-Pooling

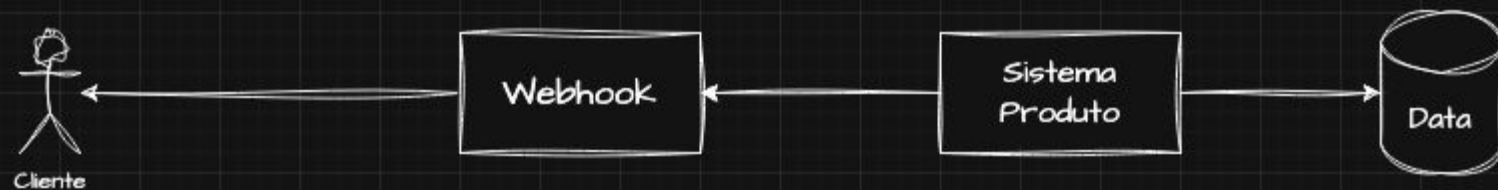
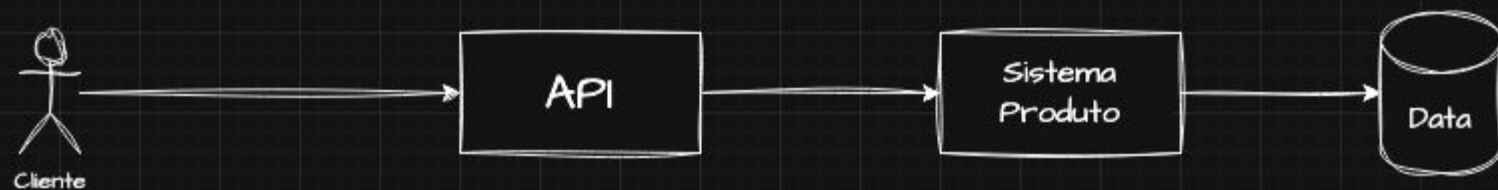
- O cliente faz requisições periódicas em intervalos fixos (ex: a cada 5s).
- Requisição fica pendurada até que o servidor tenha algo novo para entregar
- Ideal para sistemas onde o estado muda com frequência previsível.
- Simples de implementar
- Cria sobrecarga no servidor e latência perceptível entre atualizações.

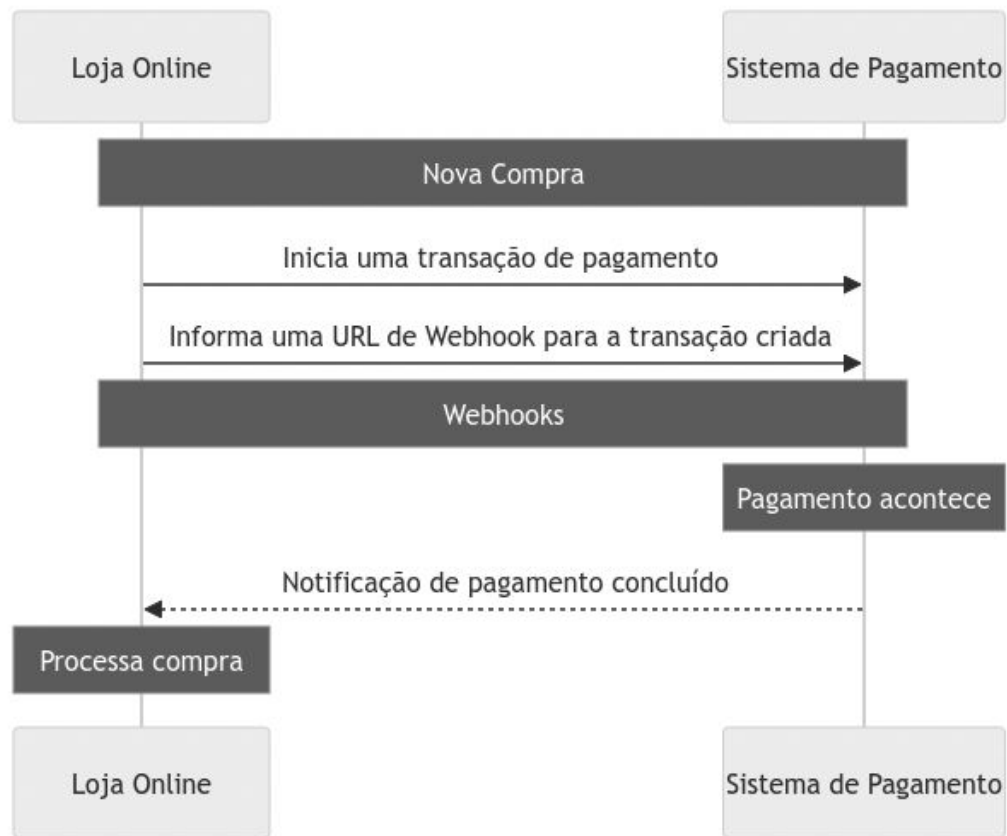
Short-Pooling

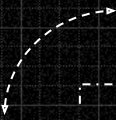


Webhooks

- Webhooks como notificações síncronas entre sistemas
- "Contrário de uma API"
- Enviar dados para os clientes
- Determinadas ações acontecerem dentro do sistema
- Atualizações contínuas sobre o estado de um recurso de interesse no servidor







5

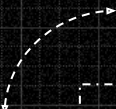
RPC

Definições e
Conceitos



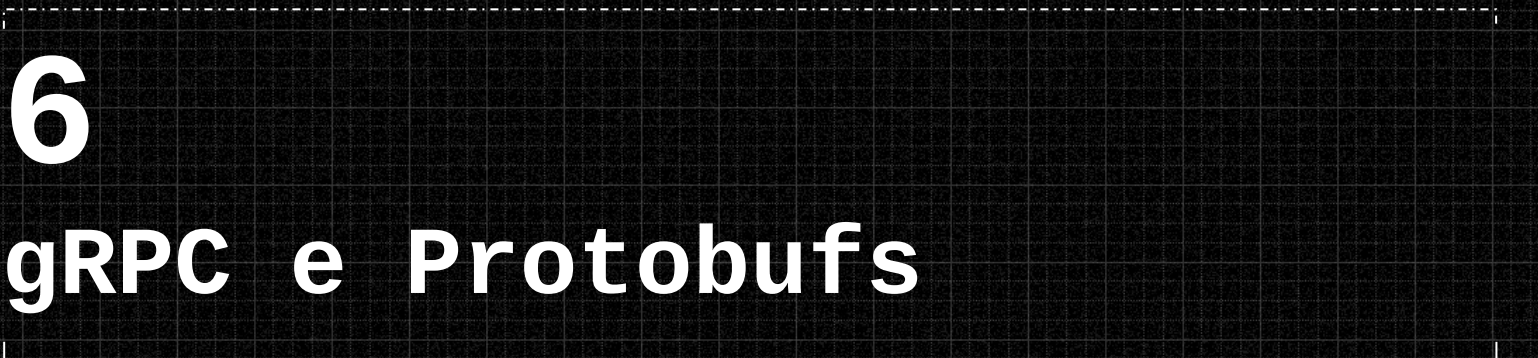
RPC - Remote Procedure Call

- Chamadas de Execução
- Chamadas para sistemas computacionais diferentes
- Abstrai a complexidade da comunicação em rede
- Permitindo aos desenvolvedores se concentrarem na lógica de negócios
- SOAP, Thrift, CORBA entre outros

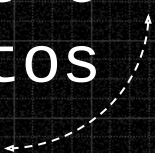


6

gRPC e Protobufs

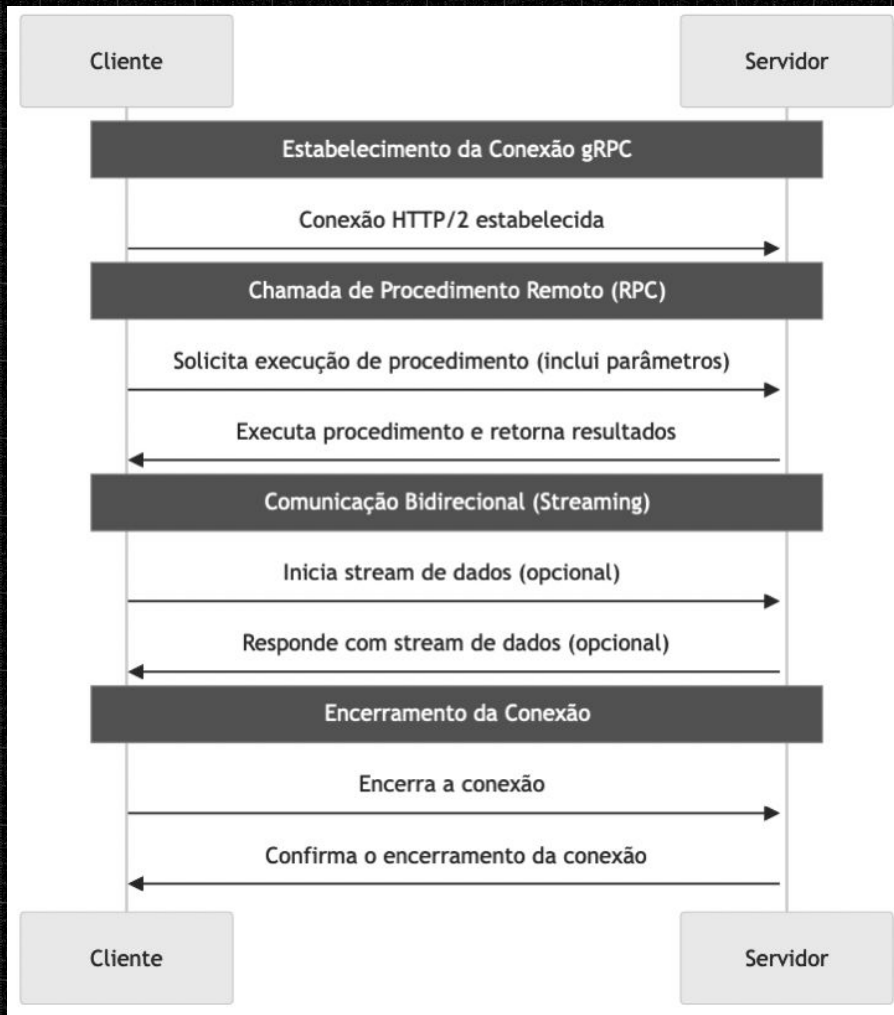


Definições e
Conceitos



gRPC - Google Remote Procedure Call

- Framework de RPC desenvolvido pela Google
- Conexão Performática e Escalável
- Design de Sistemas Distribuídos
- Performático e Escalável
- HTTP/2 e Protobufs
- Contratos Fortes
- Autenticação
- Keep Alive e Conexões Persistentes
- Balanceamento de Carga
- Múltiplas chamadas RPC em paralelo sobre uma única conexão TCP

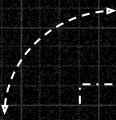


Protobufs

- Protocol Buffers
- Sistema de serialização binária
- Interface Forte de Serviços
- Estrutura de dados que serão compartilhados entre cliente e servidor
- Especificação de Métodos e Contratos

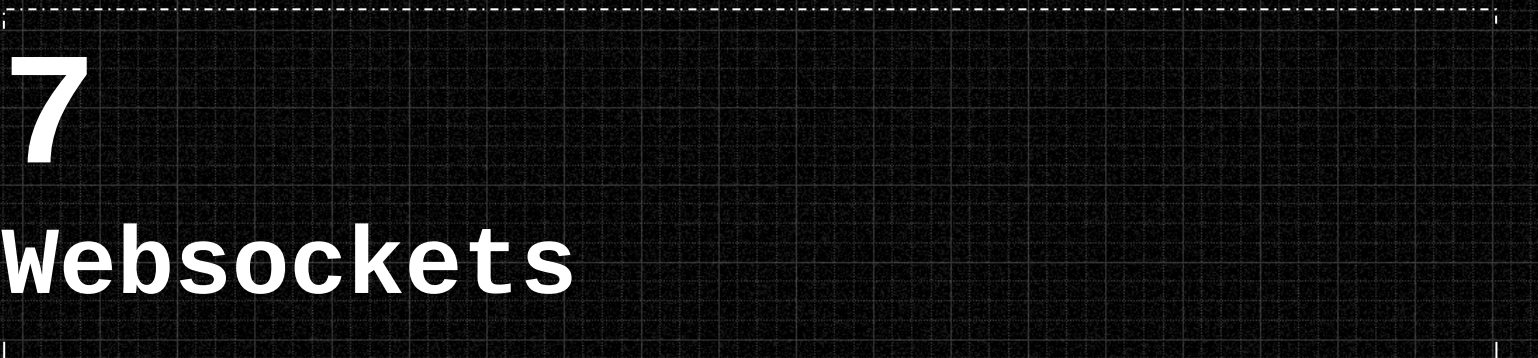
Protobufs

```
1  syntax = "proto3";
2
3  package imc;
4
5  option go_package = "service/imc";
6
7  // O serviço IMCService oferece a operação de calcular o quadrado de um número.
8  service IMCService {
9      // IMC calcula o quadrado de um número.
10     rpc Calcular (IMCRequest) returns (IMCResponse) {}
11 }
12
13 // IMCRequest contém o a altura e peso para o qual queremos calcular o IMC.
14 message IMCRequest {
15     double weight = 1;
16     double height = 2;
17 }
18
19 // IMCResponse contém o resultado do cálculo do IMC informado.
20 message IMCResponse {
21     double result = 1;
22 }
```

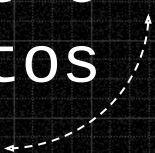


7

Websockets

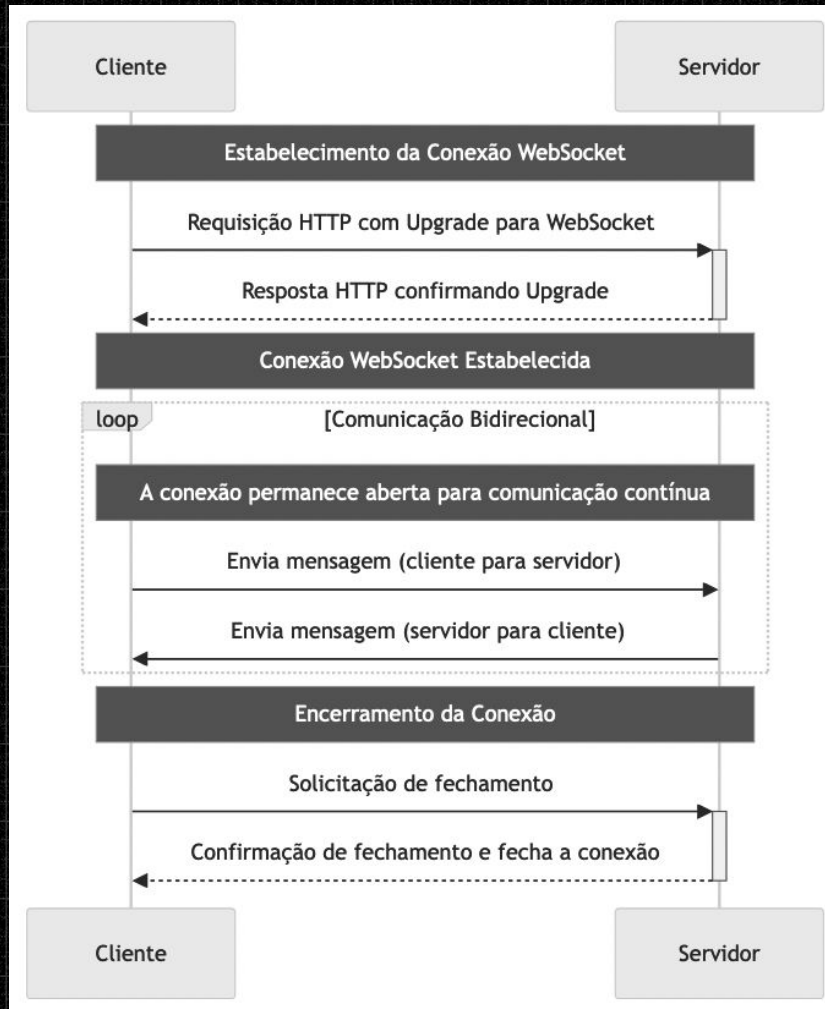


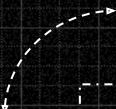
Definições e
Conceitos



Websockets

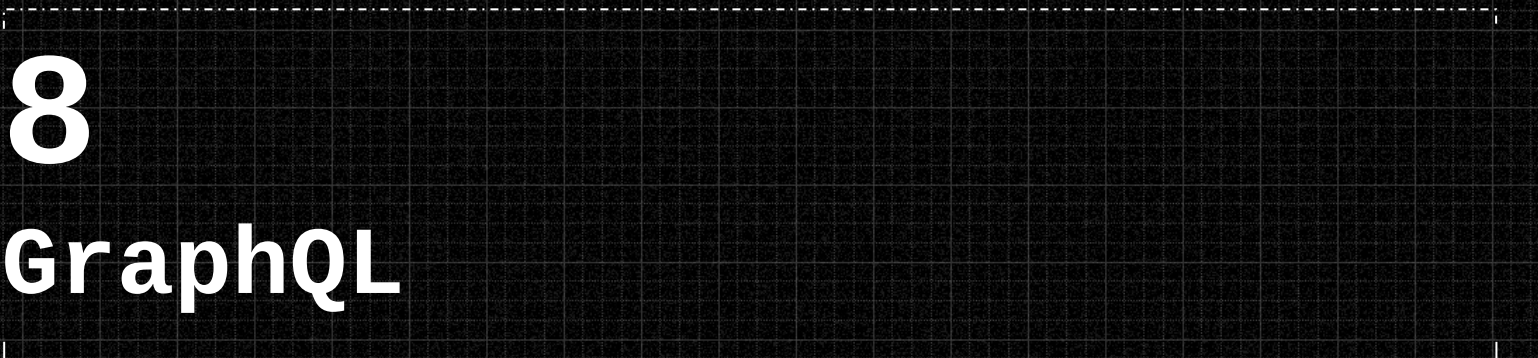
- Conexão full-duplex persistente
- Conexão vira um canal permanente
- Canal Bidirecional
- Ideal para comunicação em tempo real (chats, dashboards)
- Upgrade HTTP





8

GraphQL



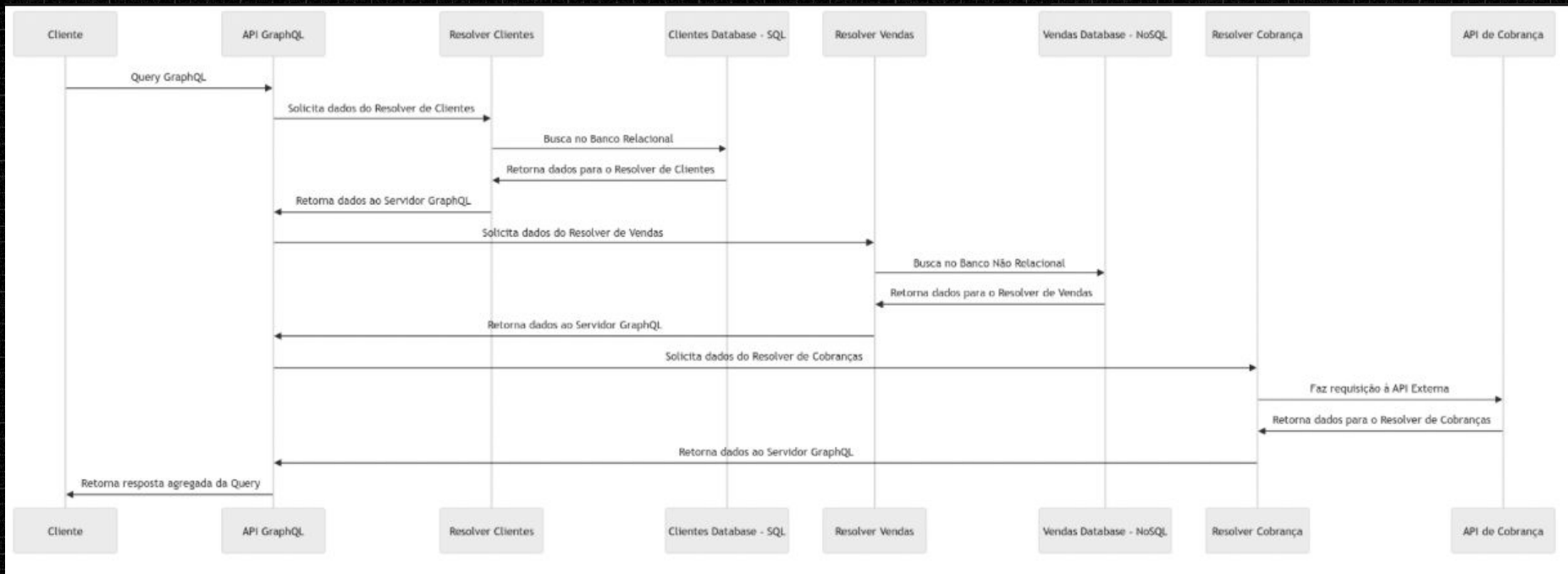
Definições e
Conceitos

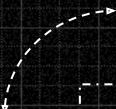


GraphQL

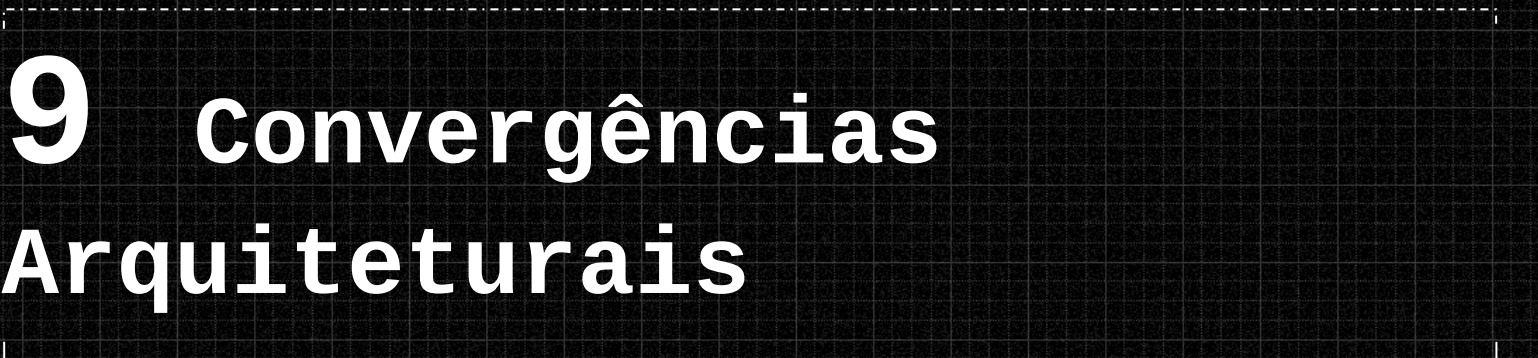
- Linguagem de consulta declarativa para APIs
- Evita over-fetching
- Evita under-fetching
- Evita alta complexidade de API's e Endpoints
- Schema SDL (Schema Definition Language)
- Query - Buscar Dados
- Mutations - Modificar Dados
- Resolvers e Múltiplos Data Sources

Resolvers e Múltiplos Data Sources



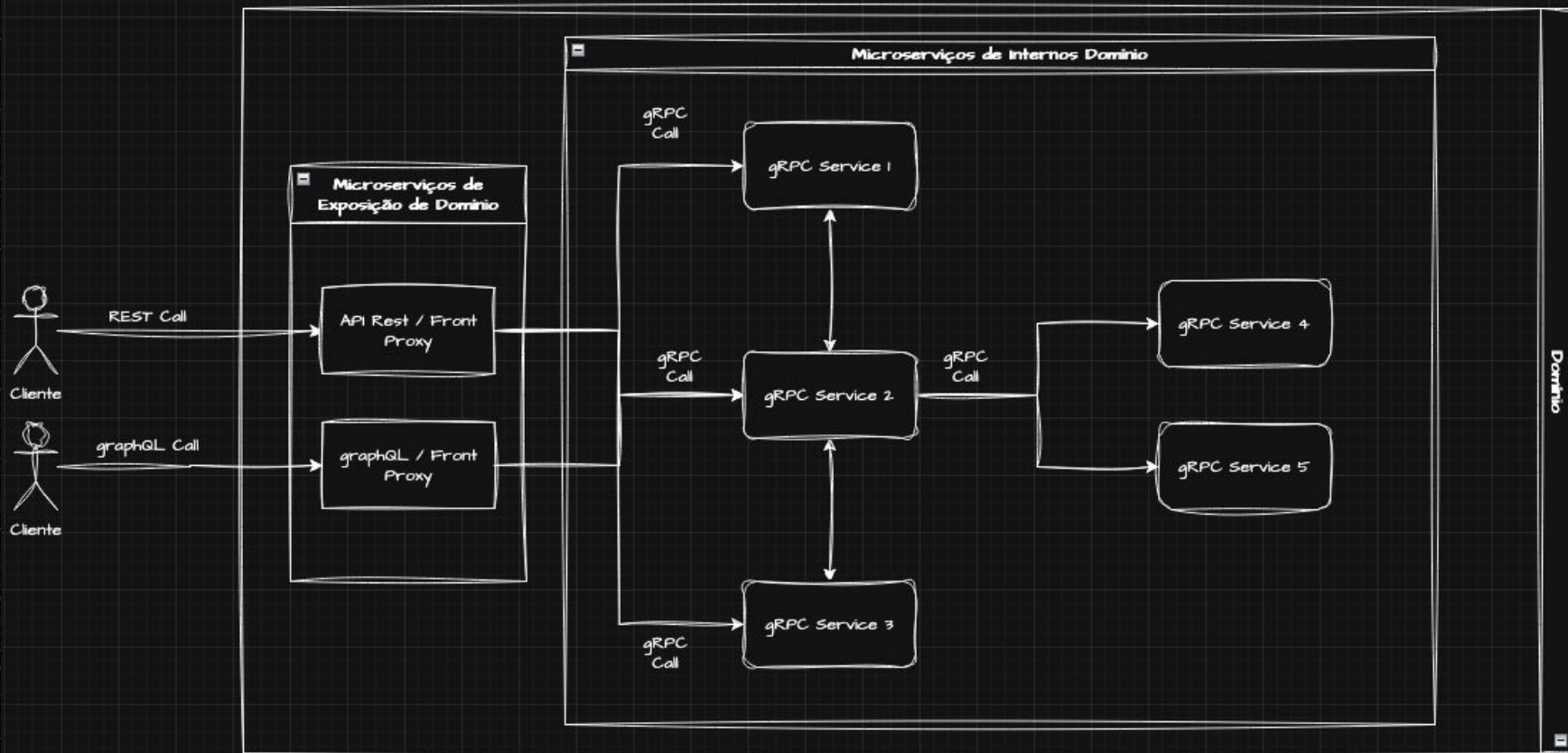


9 convergências Arquiteturais



Definições e
Conceitos





\$ whoami

Matheus Fidelis

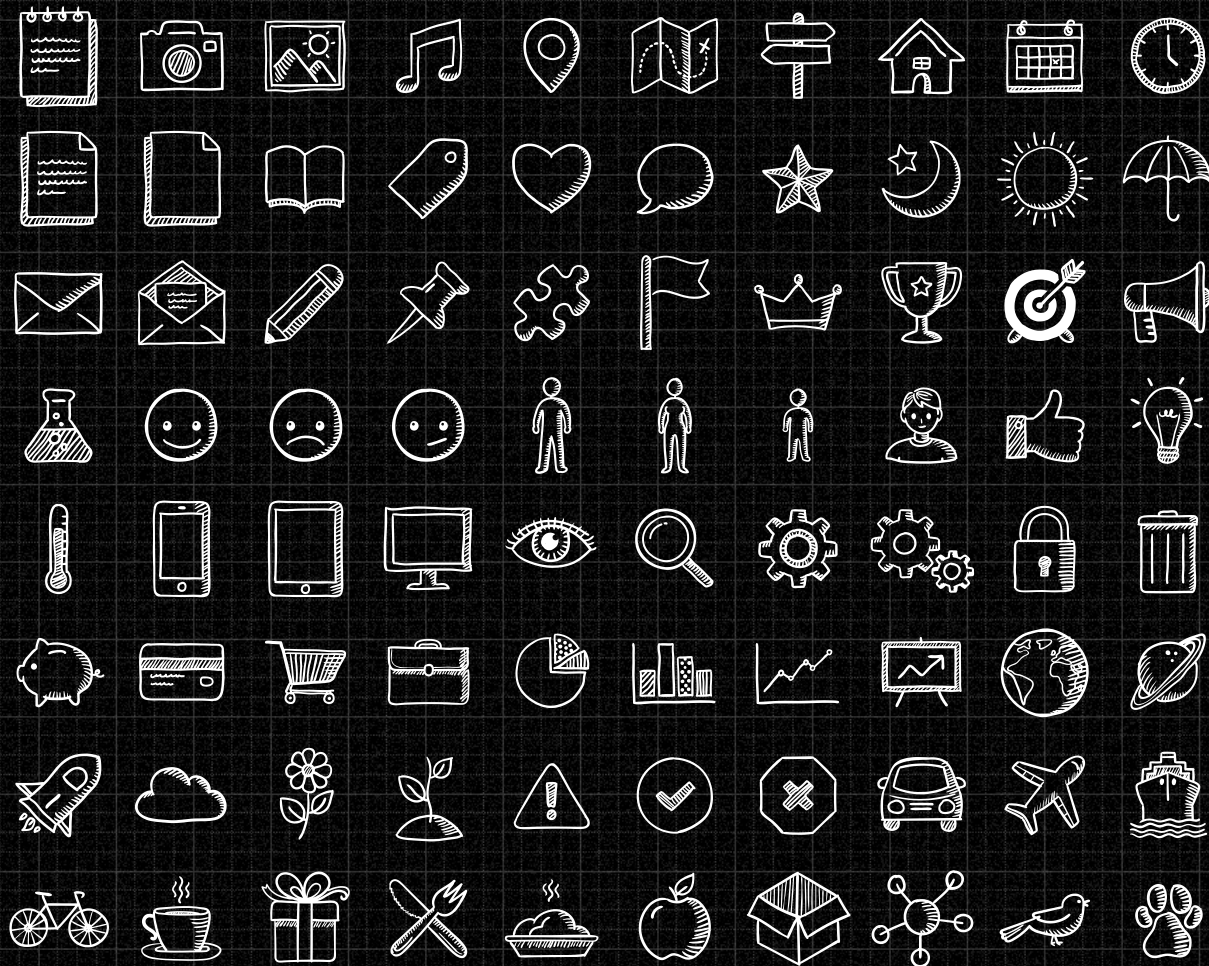
Engenheiro de \$RANDOM

@fidelissauro

<https://fidelissauro.dev>

<https://linktr.ee/fidelissauro>





SlidesCarnival icons are editable shapes.

This means that you can:

- Resize them without losing quality.
- Change fill color and opacity.

Isn't that nice? :)

Examples:

